MONASH University

MMA3001: Assignment 3

Automated Detection and Severity Scoring of Packaging Defects in Pork Rasher Trays

MMA3001 Numerical Methods and Machine Learning, Individual Project. Bryce Cecchini (34990739). 23 October 2026. Repository: https://github.com/bcec0002/MMA3001-Project

1. Engineering problem

Automated visual inspection is used on food-packaging lines to find defective products before distribution and to reduce reliance on manual inspection. This project addresses a narrow version of that task: given a top-down RGB image of a packaged pork rasher tray, (i) detect and localise packaging and product-placement defects, and (ii) convert the detections into a severity score that can support a pass/fail decision.

The problem matters because defects differ in consequence. A failed seal can compromise food safety, loose or twisted meat affects shelf life and presentation, and wrinkles affect the product's consumer appeal. A detector that only counts defects treats these equally, whereas a severity score lets trays be ranked thresholded.

Intended use: offline evaluation of stored images, not for deployment. Scope limits: one product and one fixed top-down camera view; five defect classes annotated with bounding boxes only; no human-labelled severity or pass/fail ground truth; no testing on production hardware or at line speed.

2. Inputs, outputs and data

2.1 Inputs and outputs

Item	Format, units and domain	Meaning and use
Input image	RGB JPEG, 720 × 540 px; resized (letterboxed) to 640 px on the long side	One top-down tray image

Detection	Class name; confidence 0 to 1; box as normalised x-centre, y-centre, width, height (0 to 1)	Where and what kind of defect
Defect severity	Dimensionless: class weight × fraction of image area covered by the box	Size-and-type weighted importance of one defect
Tray severity	Sum of defect severities; 0 if no defects	Tray-level score
Pass/fail	Pass if tray severity < threshold T	Inspection decision (T is an assumption)

Invalid or unsupported inputs. The label reader skips blank lines and lines with fewer than five fields and raises ValueError on non-numeric values. The severity code raises ValueError for negative or NaN box sizes, clips sizes above 1 to 1, and raises KeyError for a class with no weight. Image format and size are not validated by the code: the detector resizes inputs and unreadable files fail on load. Images from a different camera, product or lighting are outside the tested domain.

2.2 Data

The data are the Roboflow Universe project pork-rasher-error-packaging (Version 4, YOLOv8 export, licence CC BY 4.0; see References). Version 4 has 3,549 images at 720 × 540 px, derived from about 1,955 original project images, with augmentation applied to the training images. Splits and class counts are in Table 1, typical box sizes in Table 2.

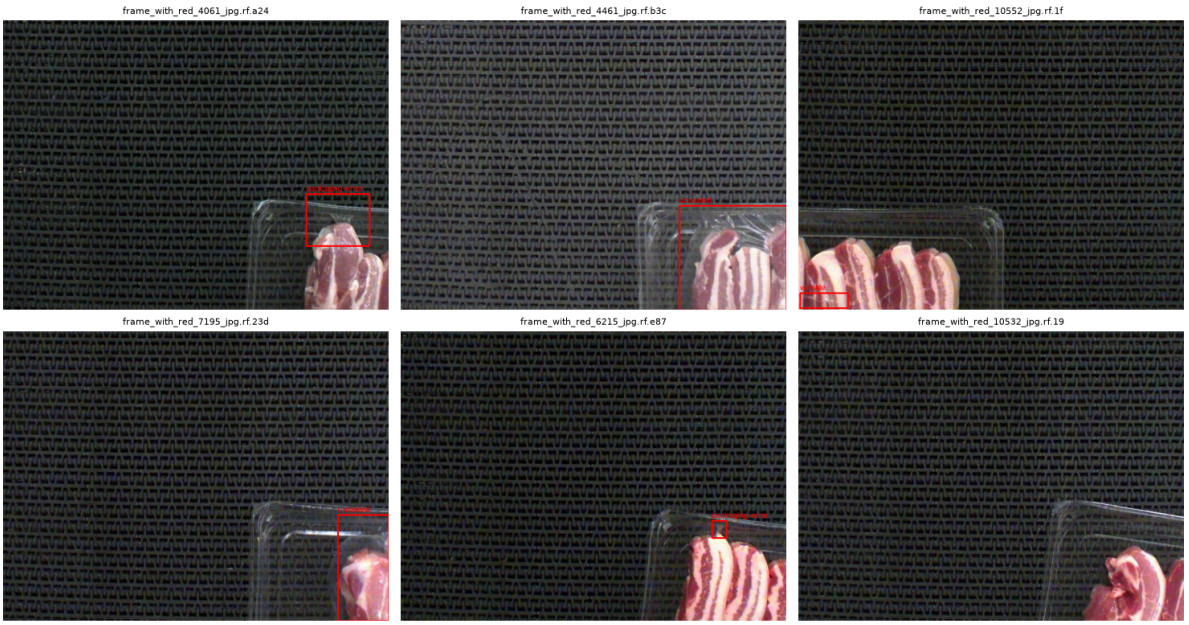


Figure 1*: six annotated sample images, produced by scripts/inspect_database.py*

Table 1. Images and boxes per split.

Split	Images	No defect	Loose meat	Packaging error	Twisted meat	unsealed	wrinkle
train	3,349	684	66	1,712	72	1,055	18
valid	120	14	3	34	3	81	0
test	80	12	3	23	4	49	0

Observations that shaped the project:

- The classes are very unbalanced: unsealed and packaging-error hold 2,767 of the 2,923 training boxes.
- Valid and test contain no wrinkle boxes and only 3 to 4 boxes each of loose-meat and twisted-meat, so those classes cannot be evaluated reliably. The test set has 79 boxes in 80 images.
- Packaging-error boxes are small (about 0.36% of the image), while unsealed boxes are large (about 10.9%).
- Many trays are cut off by the image edge, so there is no stable tray area to normalise by.
- Images may be from video frames, so near-duplicate frames may appear in more than one split.

3. Computational approach

3.1 Detector

I used YOLOv8 (Ultralytics), a single-stage detector. One forward pass of a network predicts candidate boxes with class scores at several image scales; then removes overlapping duplicates, and predictions below a confidence threshold are discarded. I fine-tuned the pretrained YOLOv8n weights (3.1 M parameters) on the training split.. Assumptions: images come from the same fixed camera set-up, annotations are correct and consistently drawn, and a bounding box is an adequate proxy for defect extent.

3.2 Settings and implementation

All runs used 30 epochs, batch size 16, seed 0, images streamed from disk, and the Ultralytics 8.4.183 defaults for everything else (full settings in results/args_*.yaml). Image size was 640 px except run C (960 px). Training and evaluation ran in Google Colab on a Tesla T4 (Python 3.13.15, torch 2.11.0+cu130). The label reader, severity code, tests and documentation were developed locally in Python 3.12 and live in src/, test/ and docs/ of the repository.

3.3 Severity scoring

I used extent only because no severity labels exist from which to fit anything richer, and larger defects often matter more. The weights are based on engineering judgement to account for this assumption.

Weights: Unsealed = 1 The loss of seal compromises the biosafety of the food meaning defects of this kind should not be undervalued.
Packaging-error =0.9 Packaging errors can potentially compromise the safety of the product but may also be little more than a cosmetic error so a value of 0.9 accounts for this whilst still being pessimistic.
Loose-meat = 0.5
Twisted meat = 0.5 both loose and twisted meat essentially cover the same defect. where the meat is not sealed by the wrapping, this poses a risk for long term shelf life but is a cosmetic defect so the weight is half of unsealed to put more focus on unsealed products.
Wrinkles = 0.2 Wrinkles are almost entirely cosmetic and matter for selling the product so companies will note a loss in sales for wrinkled goods, however the safety impact is minimal and the weight reflects this in particular.

4. Alternatives and baselines

Selection rule, decided before I saw run C's results: choose by validation AP50 on unsealed and packaging-error (the two classes with enough boxes to judge) and break ties on cost; C would replace A only if its validation packaging-error AP50 beat A's by about 0.05 without hurting unsealed. Test results for A and B were already visible when the rule was set; this is a limitation of the process.

Table 2. Comparison of runs (valid / test where two values are given).

Run	mAP50 valid / test	AP50 unsealed	AP50 packaging-error	Inference ms	Parameters	GFLOPs	Training
A: YOLOv8n, 640	0.567 / 0.522	0.921 / 0.857	0.322 / 0.325	5.0 / 5.9	3.13 M	8.6	~40 s/epoch
B: YOLOv8s, 640	0.490 / 0.408	0.924 / 0.837	0.341 / 0.603	8.5 / 8.9	11.13 M	28.4	~51 s/epoch
C: YOLOv8n, 960	0.490 / 0.378	0.927 / 0.855	0.370 / 0.406	9.7 / 8.4	3.13 M	about 19 (estimate)	0.747 h total

- A versus B: B is no better on unsealed (valid AP50 0.924 vs 0.921) or packaging-error (0.341 vs 0.322) but costs 3.6 times the parameters, 3.3 times the GFLOPs and 1.5 to 1.7 times the

time, so A is selected. B's much higher test packaging-error AP50 (0.603 vs 0.325) did not reproduce on validation; with 23 test boxes and one seed it is not evidence of a real gain.

- A versus C: C raised packaging-error AP50 by 0.048 (valid) and 0.081 (test) with no change on unsealed, but costs about 2.2 times the training time and 1.4 to 1.9 times the inference time, and its validation gain fell just short of the rule, so A is retained. Localisation quality (mAP50-95) for packaging-error stayed low (0.139 valid, 0.102 test at 960 px), so resolution alone does not fix it.

5. Validation

5.1 Design

The validation split (120 images) was used for model selection and the confidence threshold; the test split (80 images) was used to report the selected model. Detection was scored with Ultralytics' COCO-style metrics: precision, recall, AP50 and AP50-95 per class, and their mean.

5.2 Detection results (run A, test split)

Class	Test boxes	Precision	Recall	AP50	AP50-95
unsealed	49	0.659	0.869	0.857	0.589
packaging-error	23	0.354	0.565	0.325	0.085
loose-meat	3	1.000	0.563	0.806	0.304
twisted-meat	4	0.000	0.000	0.099	0.018
mean of the four	79	0.503	0.499	0.522	0.249

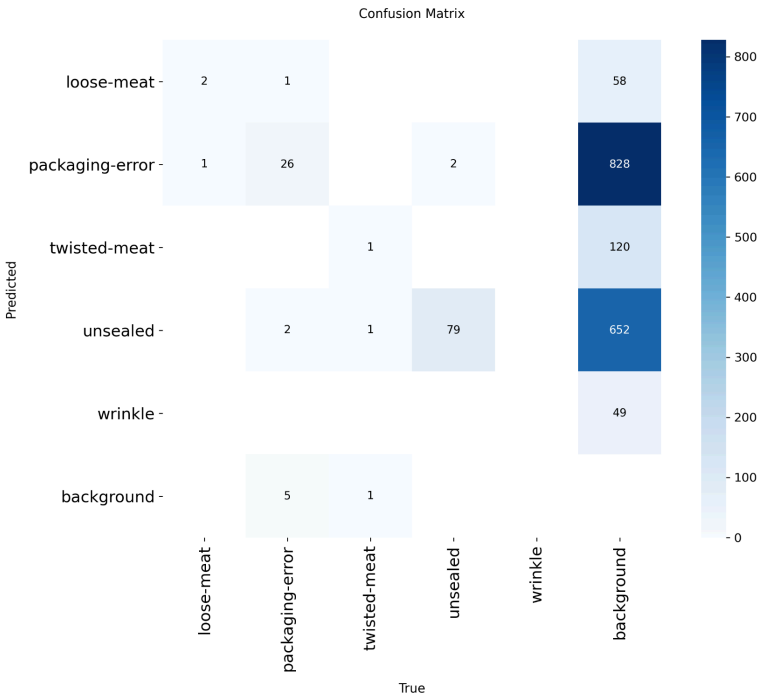


Figure 2: confusion matrix, results/confusion_matrix.png

- Unsealed (49 boxes) is detected reliably: AP50 0.857, recall 0.869.
- Packaging-error (23 boxes) is detected but poorly localised: AP50 0.325 but AP50-95 only 0.085, with low precision. Boxes of about 39 × 31 px are small enough that a few pixels of offset sharply reduce overlap, which fits this pattern; run C's modest gain partly supports it.
- Loose-meat and twisted-meat (3 and 4 boxes) cannot be judged: one detection changes recall by 25 to 33 points, and loose-meat AP50 moved from 0.359 on validation to 0.806 on test for the same model. Wrinkle has no valid or test boxes and is untested.

5.3 Severity results

Confidence threshold, chosen on the validation split (run A):

Confidence	Valid MAE	Valid Spearman
0.10	0.0282	0.762
0.25	0.0174	0.886
0.40	0.0172	0.877

0.50	0.0188	0.868

I chose 0.25: its MAE differs from 0.40 by only 0.0002 (noise on 120 images) and it has the higher rank correlation. At 0.10 the error is clearly worse, probably because low-confidence spurious boxes inflate tray severity (not tested).

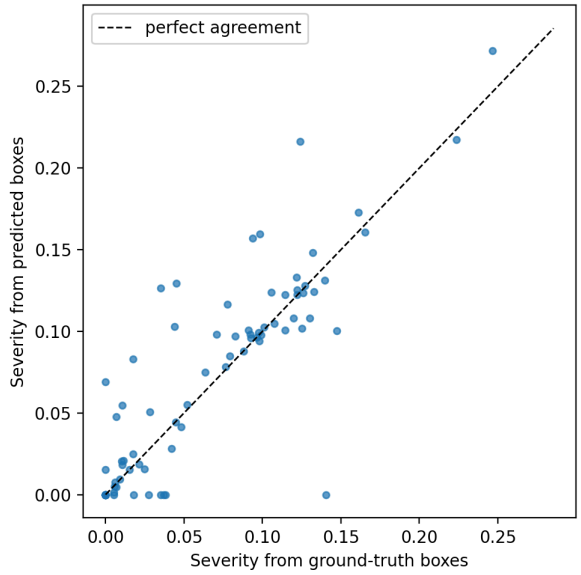


Figure 3: predicted vs ground-truth tray severity, results/severity_scatter_test.png

Pass/fail. No ground-truth pass/fail labels exist, so no threshold is correct; the choice is a policy trade-off. A severity of 0.05 corresponds to about half of a typical unsealed defect. Accuracy is misleading at low thresholds (at 0.005, 68 of 80 trays fail by ground truth, so failing every tray would be 85% accurate), so Table 4 reports recall and false-fail rate. Counts of caught and missed trays are approximate, derived from the recall values.

Table 3. Test-set pass/fail agreement at confidence 0.25.

Threshold	**Fail / pass by ground truth**	**Fails caught**	**Fails missed**	**False fails**	**False-fail rate**
0.005	68 / 12	~59 (0.87)	~9	2	17%
0.010	61 / 19	~55 (0.90)	~6	3	16%

0.020	53 / 27	~46 (0.87)	~7	7	26%
0.050	40 / 40	~39 (0.97)	~1	7	18%
0.100	23 / 57	~22 (0.96)	~1	7	12%

These rest on 23 to 68 failing trays (for example 22 of 23 caught has a 95% interval of roughly 0.8 to 1.0), and the three counts of 7 false fails are small numbers, so differences between thresholds should not be over-read.

5.4 Failure analysis

Table 4 lists the five test images with the largest severity error (Figure 4; green = ground truth, red = prediction).

Image (frame_with_red_)	**Severity GT → predicted**	**Observation**
3117	0.141 → 0	Large labelled unsealed box on a tray cut off by the left edge; nothing predicted (missed defect)
102	0.124 → 0.216	Two overlapping trays; the predicted box is larger than the labelled one
8208	0.035 → 0.127	Small labelled box at the tray edge; several overlapping, larger predicted boxes
6338	0.045 → 0.130	Same pattern: multiple overlapping predicted boxes
7471	0 → 0.069	No labelled box; a predicted box over the tray contents (false alarm or missing label)

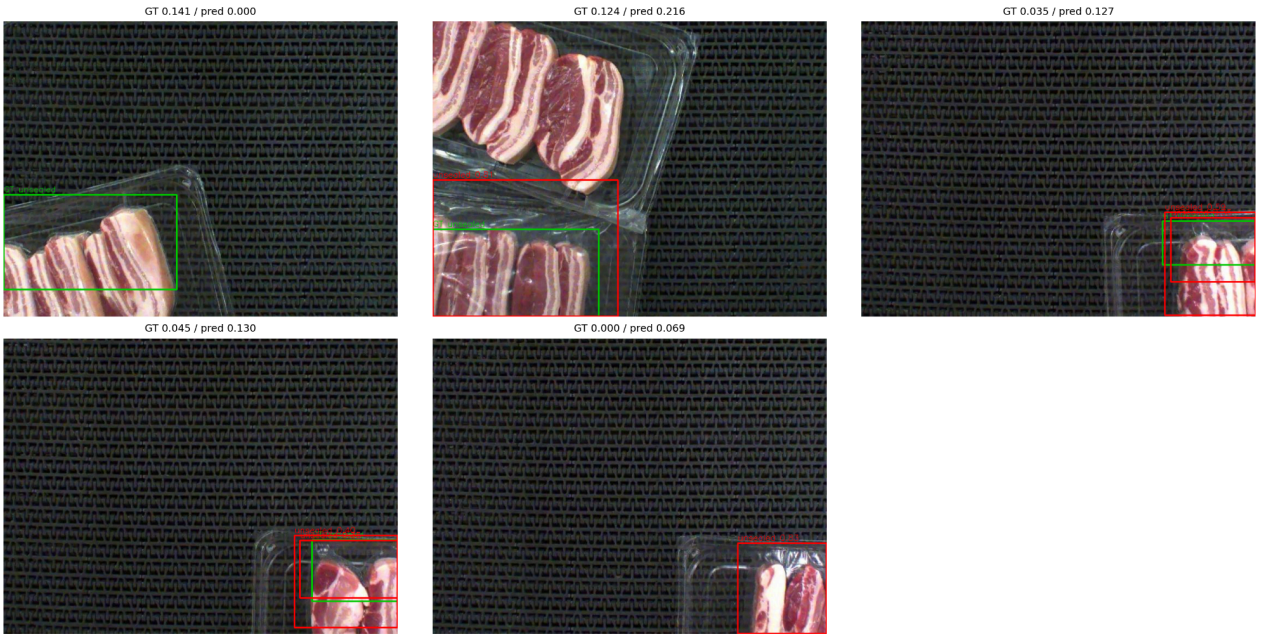


Figure 4: five worst severity cases, results/severity_worst5_test.png

Three of the five are overestimates from oversized or duplicated boxes. Because tray severity sums areas, overlapping boxes on one defect are counted twice. Merging overlapping same-class boxes or using the union area would remove this (not tested). Trays cut off by the frame or overlapping another tray appear in two of the cases.

5.5 Code verification

All 16 pytest tests pass: 4 for the label reader (valid lines, empty file, skipped short lines, error on non-numeric values) and 12 for severity (hand-checkable extents such as a half-width, half-height box giving 0.25, clipping, invalid sizes, class weighting, unknown class, tray sums and an empty tray, threshold behaviour, and a check that the weights cover all five dataset classes). The report is results/pytest_report.html. A fresh clone installed from requirements.txt passes the same 16 tests with no dataset present.

6. Optimisation and performance

Timing and model size come from Ultralytics' built-in timing and model summary on a Colab T4. I did not use a separate profiler, measure GPU memory or estimate arithmetic intensity, which limits this analysis.

- Runtime: run A takes about 5.0 to 5.9 ms per image for inference and about 14 ms end to end including pre- and post-processing (2.9 + 5.9 + 5.4 ms on test), roughly 70 images per second on a T4. Repeated evaluations vary (run B preprocessing was 10.3 ms on test and 2.7 ms on validation), so differences under 1 to 2 ms are not meaningful. Whether this is fast enough depends on line throughput and hardware, neither of which was tested.
- Model cost: A has 3.13 M parameters and 8.6 GFLOPs; B 11.13 M and 28.4 GFLOPs; C has A's parameters and about 19 GFLOPs at 960 px (estimated as 2.25 × 8.6 because convolution cost scales with pixel count; the logged 8.6 was measured at 640 px).

7. Limitations and lessons learned

Limitations are set out in Sections 2.2, 3.3 and 5.6: unbalanced data and untestable rare classes, an assumed severity definition and weights with no severity ground truth, no classical baseline, single seeds, and area-summing that inflates severity when boxes overlap.

Lessons:

1. Inspect the data first. Counting boxes and measuring box sizes before training exposed the imbalance, the missing wrinkle boxes in valid and test, and the small packaging-error boxes, and these shaped later decisions.
2. Aggregate metrics can mislead. The headline mAP50 averaged over classes with 3 to 4 boxes; per-class results with instance counts were far more informative.
3. Fix the selection rule before looking at results and select on the validation split. Run B's large test-split gain disappeared on validation.

Future work: a weight sensitivity analysis; union area or merged boxes for severity; multiple seeds and bootstrap intervals for detection metrics; a classical baseline; more rare-class data and a check for near-duplicate frames; severity labels from inspectors; per-class thresholds.

8. AI-use reflection

Tool. Claude (Anthropic), used throughout in chat.

What it was used for. Turning the project brief and dataset choice into a workflow; guiding the set-up of GitHub, VS Code, Colab and Google Drive; drafting the code in src/, the tests and the Colab notebook cells; suggesting the repository structure and documentation; explaining the concepts behind each block of code and what the printed results show; and drafting the structure and wording of this report and the readme.

Level of contribution. Claude wrote almost every line of code and the first drafts of the documentation. I ran everything, recorded the outputs, and made minor edits to the code. Claude also made templates for the readme and report, and suggested where to find documentation for the licence.

Why AI was used. AI was used to quickly fill in the gaps in my knowledge, starting this project I was very unfamiliar with python and colab and especially training data. I attempted to extract as much information from the AI as possible so I could be confident in explaining every step.

How AI output was checked. In order to verify the work was valid, I ran the 16 pytest tests which can be checked by hand. I also checked the spit counts against the dataset numbers. The worstcase images were also checked by me in order to get an idea of why they performed so poorly. I tested the repo from a fresh clone.

Decisions that remained mine. The dataset and problem; limiting severity to box extent rather than normalising by class; the class weights and their justification; the final model; and what is claimed in this report.

Errors and unhelpful suggestions. An early reading, from a 1-epoch run, had packaging-error missed outright, was corrected after 30-epoch results showed it was found but poorly localised and the severity cell used YOLO without importing it after a notebook reset.

References

Jocher, G., Chaurasia, A. and Qiu, J. (2023). Ultralytics YOLOv8 [software]. https://github.com/ultralytics/ultralytics

Hello (2025). Pork Rasher Error(Packaging) Dataset [open source dataset]. Roboflow Universe, Version 4, licence CC BY 4.0. https://universe.roboflow.com/hello-2aqe0/pork-rasher-error-packaging (visited 2026-10-23)

Appendices

A. Repository structure and reproduction steps (see README). B. Run settings (results/args_*.yaml). C. Pytest report (results/pytest_report.html).